

Crypto Vault — Technical documentation

Implementation specification, security model and declared limits

Document version: 1.1 Reference: `crypto-vault.html` — SHA-256 hash published with every release

Preliminary notice

This document describes how Crypto Vault works technically. It is not a promotional document and it contains no security guarantees: the statements made here are verifiable by reading the source code, reproducing the build and running the tests described in section 8.

Crypto Vault is free software distributed under the MIT licence, **without any warranty**, express or implied. Custody of keys and backups is the sole responsibility of the user. The authors have no access to the wallets generated, keep no data and can under no circumstances recover seeds, passphrases or funds.

The software does not constitute financial, legal or tax advice. Users are responsible for verifying that their use complies with the regulations applicable in their jurisdiction.

1. Object and scope

Crypto Vault is a web application contained in a single HTML file that generates cryptographic keys for cryptocurrency wallets according to public standards, and derives their public addresses.

1.1 Implemented functions

Function	Reference standard
Mnemonic phrase generation	BIP-39
Hierarchical deterministic derivation	BIP-32
Derivation paths	BIP-44, BIP-49, BIP-84, BIP-86, BIP-48
Taproot addresses	BIP-341
Multisig key ordering	BIP-67
Interoperable threshold backup	SLIP-39
ed25519 curve derivation	SLIP-10
Binary representation of the backup	No standard: scheme documented in 5.4
Coin identifiers	SLIP-44

1.2 Deliberately unimplemented functions

- **Transaction signing.** The software does not build, sign or broadcast transactions. Signing would require connectivity or interaction with nodes, which is incompatible with the isolation model adopted here.
- **Balance lookups.** No queries to blockchains, explorers or third-party services.
- **Persistence.** No writes to `localStorage`, `sessionStorage`, `IndexedDB`, cookies or the filesystem. Cryptographic material exists exclusively in the volatile memory of the browser tab and ceases to exist when the page is closed.
- **Telemetry.** No usage data collection, no identifiers, no network requests at runtime.

2. Architecture

2.1 Distribution model

The product is a single self-contained HTML file. All cryptographic libraries are embedded into the file at build time. At runtime no external resource is requested: no CDN, no remote fonts, no external images.

Verifiable consequence: the file behaves identically on a device that has never had connectivity.

2.2 Build chain

```
src/app.js      application code
src/index.html  structure and stylesheets
|
├─ esbuild (bundling, minification, ES2020 target)
|   └─ alias: crypto → src/crypto-shim.js
|      └─ inject: src/buffer-shim.js
```

```
|
└─ scripts/build.js
    └─ dist/crypto-vault.html
```

Dependencies are pinned to exact versions in `package-lock.json`. The `npm ci` command installs exactly those versions, making the build reproducible.

2.3 Cryptographic libraries

Library	Use
@scure/bip39	Mnemonic encoding and validation, PBKDF2
@scure/bip32	Hierarchical deterministic derivation
@noble/hashes	SHA-256, SHA-512, RIPEMD-160, HMAC, PBKDF2
@noble/curves	secp256k1 arithmetic (required for Taproot)
@noble/ed25519	ed25519 curve (Solana)
slip39	SLIP-39 splitting and reassembly
bech32	bech32 and bech32m encoding
bs58	Base58 encoding

The `@noble` and `@scure` families are dependency-free implementations, independently audited and widely adopted across the ecosystem.

2.4 Browser environment adapters

The `slip39` library is written for Node.js and uses the `crypto` module and the `Buffer` global, both absent in browsers. Two minimal adapters were written:

- `src/crypto-shim.js` — exposes `randomBytes` (backed by `crypto.getRandomValues()`), `pbkdf2Sync` and `createHmac` (backed by `@noble/hashes`). No other function is exposed.
- `src/buffer-shim.js` — exposes `Buffer.from` and a few helper functions, returning `Uint8Array`.

The adapters were validated by running the entire official SLIP-39 vector suite through them (section 8.3).

2.5 Language

The interface is in English only. There is no runtime translation layer and no localisation catalogue: every string that appears on screen is written directly in the source, which keeps the shipped file smaller and removes an entire class of “untranslated string” defects.

3. Entropy generation

3.1 Requirement

The security of a deterministic wallet depends entirely on the unpredictability of the initial value. A faulty generator makes the robustness of every subsequent operation irrelevant. Real cases of fund loss caused by malfunctioning pseudo-random generators are documented.

3.2 Sources

Primary source — system CSPRNG. `crypto.getRandomValues()`, the standard interface exposed by the browser and fed by the operating system generator (`/dev/urandom` on Linux and macOS, `BCryptGenRandom` on Windows). 64 bytes are requested.

Secondary source — typing dynamics. For every keystroke the key code and a high-resolution timestamp are recorded. The relevant entropic contribution lies in the inter-keystroke intervals, not in the characters. Minimum thresholds enforced: 20 characters, 5 seconds, 10 distinct keys.

Tertiary source — pointer dynamics. Sampling of coordinates and timestamps during pointer movement. Progress is conditional on four criteria being satisfied jointly: minimum time (8 seconds), number of events, distance travelled and direction changes. A fast straight-line movement does not satisfy the criteria.

Quaternary source (optional) — physical dice rolls. A sequence entered manually by the user. Each roll of a six-sided die contributes $\log_2 6 \approx 2.585$ bits. The number of rolls required is computed from the selected seed length. This is the only source generated entirely outside the computer system.

3.3 Combination

```
entropy = SHA-256( CSPRNG(64) || H(keyboard) || H(mouse) || H(dice) )
```

For lengths above 32 bytes the function is iterated with a domain counter.

Three implementation invariants:

1. The CSPRNG contribution is **always present and never conditional**.
2. The additional sources are **additive**: they never replace or disable the CSPRNG.
3. Given the properties of the hash function used, the result is no weaker than the strongest of the input sources.

3.4 Statistical checks

Before use, the generated entropy is subjected to three checks. The failure of any one of them **stops generation**, with no fallback to weaker modes.

Check	Failure condition
Constant value	All bytes identical
Repeatability	Two consecutive calls produce identical output
Balance (monobit)	Fraction of bits set to 1 outside the range [0.25 – 0.75]

At start-up the availability and operation of the CSPRNG is also verified. If the check fails, the generation function stays disabled.

Declared limit. These checks detect macroscopic failures. No software check can measure the actual entropy of a sequence: an output with degraded but statistically plausible entropy would not be caught.

4. Key derivation

4.1 BIP-39

The entropy is extended with a checksum equal to the first $n/32$ bits of its own SHA-256, split into groups of 11 bits, and each group indexes a word in the official English dictionary of 2048 words.

Words	Entropy	Checksum
12	128 bits	4 bits
15	160 bits	5 bits
18	192 bits	6 bits
21	224 bits	7 bits
24	256 bits	8 bits

The 64-byte binary seed is obtained through PBKDF2-HMAC-SHA512 with 2048 iterations and salt "mnemonic" || passphrase.

Only the English dictionary is used. The localised dictionaries, although part of the standard, have uneven support in destination wallets and would introduce a concrete risk of unrecoverability.

4.2 Derivation paths

Network	Path	Standard
Bitcoin — Native SegWit	m/84'/0'/0'/0/i	BIP-84
Bitcoin — Taproot	m/86'/0'/0'/0/i	BIP-86
Bitcoin — P2SH-SegWit	m/49'/0'/0'/0/i	BIP-49
Bitcoin — Legacy	m/44'/0'/0'/0/i	BIP-44
Bitcoin — P2WSH multisig	m/48'/0'/0'/2'	BIP-48
Ethereum and EVM networks	m/44'/60'/0'/0/0	BIP-44
TRON	m/44'/195'/0'/0/0	SLIP-44
Solana	m/44'/501'/0'/0'	SLIP-10

4.3 Bitcoin address construction

- **Legacy (P2PKH):** Base58Check(0x00 || RIPEMD160(SHA256(pubkey)))
- **P2SH-SegWit:** redeem script 0x0014{hash160(pubkey)}, address Base58Check(0x05 || hash160(redeem))
- **Native SegWit (P2WPKH):** bech32, witness v0, program hash160(pubkey)
- **Taproot (P2TR):** x-only internal key P; tweak t = tagged_hash("TapTweak", P); output key Q = P + tG; bech32m encoding, witness v1. No script tree (key-path spend, BIP-86).

4.4 Multisig

Script 0P_m <pubkey...> 0P_n 0P_CHECKMULTISIG, wrapped in P2WSH. Public keys are sorted lexicographically according to BIP-67, which makes the resulting address independent of the order in which they were entered.

The descriptor produced has the form `wsh(sortedmulti(m,[fingerprint/48h/0h/0h/2h]xpub.../0/*,...))` and can be imported into wallets that support the descriptor standard.

In the mode where all keys are generated locally, each key derives from an independent invocation of the entropy combination function, with a fresh draw from the CSPRNG. The resulting keys are therefore statistically independent.

5. Backup splitting

5.1 Sequential split

Partitioning of the word sequence into consecutive groups. No cryptographic transformation. Reassembly is manual and requires no software.

Property: knowledge of a proper subset of the groups reduces the search space in proportion to the known words. It offers no theoretical secrecy guarantee; it offers resistance to partial discovery and independence from any tool.

5.2 Internal threshold scheme (Shamir)

Shamir Secret Sharing applied to the BIP-39 entropy.

- Finite field $GF(2^8)$, irreducible polynomial `0x11b`
- Exponential and logarithm tables built with **generator 3**
- Lagrange interpolation for reconstruction
- Each share is converted back into a valid BIP-39 mnemonic
- Verification code: first 2 bytes of the SHA-256 of the original entropy, in hexadecimal

Implementation note. Generator 2 is not primitive with respect to the polynomial `0x11b`: it generates a subgroup of 51 elements instead of the 255 required. Using generator 3 is a correctness condition of the scheme.

Security property. With fewer shares than the threshold, the distribution of the secret remains uniform: no information is revealed. This is perfect secrecy in the information-theoretic sense, not computational hardness.

Declared limit. The scheme is not an interoperable public standard. Reassembly requires this software or an equivalent implementation. The file should be kept together with the shares.

5.3 SLIP-39

Implementation conforming to the SLIP-39 specification, through the `slip39` library and the adapters described in 2.4.

- Dedicated dictionary of 1024 words; shares of 20 or 33 words
- The first three words encode the identifier and parameters and are identical across the shares of the same set
- The master secret is encrypted with the passphrase before splitting

Substantive difference from BIP-39. The reassembled master secret is used **directly** as the seed for generating the BIP-32 root key, without going through PBKDF2. It follows that, for the same initial entropy, BIP-39 and SLIP-39 produce distinct wallets. The shares of the two schemes are in no way interchangeable.

Passphrase behaviour. In accordance with the specification, a wrong passphrase produces no error: it generates a different master secret and therefore a distinct, empty wallet.

5.4 Binary representation (powers-of-2 backup)

Every BIP-39 word is identified by its index in the dictionary, represented as a sum of powers of two across twelve columns (2048 ... 1).

Choice of numbering. The internal BIP-39 index is zero-based; the printed document instead uses the numbering 1–2048. With zero-based numbering the first word of the dictionary (`abandon`) would have the value 0, and the corresponding row would carry no marks, indistinguishable from a row that has not been filled in yet. One-based numbering removes this ambiguity: no row is ever empty.

Correctness verified across all 2048 words, both encoding and decoding.

The document produced shows neither the words nor the numeric indexes: it contains only the grid. Conversion requires the numbered list of BIP-39 words, which is also printable and public by nature.

6. Security model

6.1 Assumptions

The model assumes that:

- the device on which the software runs is not compromised;
- the file executed matches the published one, verifiable through the SHA-256 hash;

- the JavaScript engine and the browser CSPRNG behave according to their specifications.

6.2 Threats addressed

Threat	Countermeasure
Network exfiltration	Total absence of network code
Unintended persistence to disk	No storage APIs used
Faulty or manipulated CSPRNG	Mixing with independent sources, including sources outside the system (dice)
Macroscopic generator failure	Blocking statistical checks
Discovery of the paper backup	Passphrase, threshold splitting
Partial loss of the backup	Threshold schemes (Shamir, SLIP-39)
Transcription error	BIP-39 checksum, verification code, double-check function
Compromise of one signing device	Multisig with keys on separate devices

6.3 Threats not addressed

The model does **not** protect against:

- a compromised operating system, keyloggers, malware with access to process memory;
- physical compromise of the device during or after generation;
- observation of the screen (video recording, reflections, bystanders);
- user error in keeping the backups;
- loss of the passphrase, which makes the funds permanently inaccessible;
- vulnerabilities present in the browser engine or in the embedded libraries;
- defects in the entropy combination logic, which no number of additional sources could compensate for.

For these reasons the user-facing documentation recommends running the tool on a system isolated from the network, preferably booted from removable media and without persistence.

7. Personal data

The software does not collect, process or transmit personal data. There are no servers, endpoints or recipients.

All cryptographic material lives in the volatile memory of the browser tab for the duration of the session. Closing the page ends it. No files, cookies or local storage entries are created.

The printable documents generated by the software are produced locally and never pass through any external service. Those documents are deliberately free of headers, marks, dates and explanatory text, to limit what could be inferred from an accidental discovery.

8. Verifiability

Every statement in this document can be verified independently.

8.1 Integrity of the distributed file

```
sha256sum crypto-vault.html           # Linux, macOS
Get-FileHash crypto-vault.html -Algorithm SHA256 # Windows
```

The value must match the one published in the `SHA256SUMS` file of the release.

8.2 Build reproducibility

```
git clone <repository>
cd crypto-vault
npm ci
npm run build
sha256sum dist/crypto-vault.html
```

8.3 Test vectors

```
npm test
```

BIP-39 / BIP-32. Canonical mnemonic `abandon × 11 + about` :

Network	Expected address
Bitcoin Native SegWit	bc1qcr8te4kr609gcawutmrza0j4xv80jy8z306fyu
Bitcoin Taproot	bc1p5cyxnuxmeuwuvkwfem96lqzszo2n6xdcjrs20cac6yqj jwudpxqkedrcr
Bitcoin P2SH-SegWit	37VucYSaXLCAsxYyAPfbSi9eh4iEcbShgf
Bitcoin Legacy	1LqBGSKuX5yYUonjxT5qGfpUsXKYYWeabA
Ethereum	0x9858EfFD232B4033E47d90003D41EC34EcaEda94
Solana	HAgk14JpMQLgt6rVgv7cBQFJWFto5Dqxi472uT3DKppk

SLIP-39. The 45 official vectors of the `trezor/python-shamir-mnemonic` project all pass, including the resulting extended private key. Verification was performed through the browser adapters described in 2.4, not through native Node.js APIs.

Internal threshold scheme. Verified by reconstruction from random subsets of cardinality equal to the threshold, and by failure with lower cardinality.

8.4 Verifying the absence of network traffic

Run the file with a traffic analyser active, or on a physically disconnected device. No outbound packet should be observed during the entire generation and derivation cycle.

8.5 Cross-checking

Import the same mnemonic into an independent implementation (Sparrow, Electrum) and compare the derived addresses. Agreement between independent implementations is the most significant verification available to an end user.

8.6 Code inspection

The application code lives in `src/app.js`. Elements verifiable by direct inspection:

- absence of `Math.random()` in cryptographic contexts;
- absence of `fetch`, `XMLHttpRequest`, `WebSocket`, `EventSource`, `navigator.sendBeacon`;
- absence of `localStorage`, `sessionStorage`, `indexedDB`, `document.cookie`;
- derivation paths declared in plain text for each network.

9. Declared limits

Consistently with the absence of any warranty, the following is stated explicitly.

No independent audit. The code has not undergone a paid professional security review. Passing the official vectors attests to the correctness of the derivations, not to the absence of vulnerabilities.

No operational track record. The project does not have the years of use on significant value that characterises established implementations. That experience cannot be replaced by design arguments.

Limited reach of the statistical checks. See section 3.4.

Dependence on the internal threshold scheme. See section 5.2. The SLIP-39 scheme does not have this limitation.

Dependence on the execution environment. See section 6.3.

For significant amounts, the combined use of dedicated hardware devices and multisig configurations with keys generated by heterogeneous tools is recommended.

10. Licence and references

The software is distributed under the MIT licence. The full text is in the `LICENSE` file of the repository.

Reference specifications

- BIP-32 — Hierarchical Deterministic Wallets
- BIP-39 — Mnemonic code for generating deterministic keys
- BIP-43 / BIP-44 / BIP-49 / BIP-84 / BIP-86 — Purpose field and derivation schemes
- BIP-48 — Multi-Script Hierarchy for Multi-Sig Wallets
- BIP-67 — Deterministic Pay-to-script-hash multi-signature addresses
- BIP-341 — Taproot: SegWit version 1 spending rules
- SLIP-10 — Universal private key derivation from master private key
- SLIP-39 — Shamir’s Secret-Sharing for Mnemonic Codes
- SLIP-44 — Registered coin types for BIP-0044

Security reports. Vulnerabilities should be reported following the procedure described in the `SECURITY.md` file, privately and before any public disclosure.